

Comparative Analysis of Performance Aspects Between Chroma and Pgvector as a Vector Database

Ali Zaenal Abidin¹⁾, Mychael Maoeretz Engel^{2)*}

¹⁾²⁾ Informatics Engineering, Faculty of Information Technology, Universitas Ciputra, Surabaya, Indonesia

¹⁾azaenalabidin@ciputra.ac.id

²⁾mychael.engel@ciputra.ac.id

Article history:

Received 26 Oct 2025;
Revised 30 Oct 2025;
Accepted 10 Nov 2025;
Available online 10 Dec 2025

Keywords:

ChromaDB
Performance
PostgreSQL
PGVector
Resource Efficiency

Abstract

System architects face a critical choice between specialized vector databases like ChromaDB and general-purpose options like PostgreSQL with the PGVector extension. This decision profoundly impacts TCO and system viability, yet holistic performance data under resource constraints is scarce. We answer whether a specialized or generalized architecture provides superior operational efficiency and accuracy when resources are limited and providing an evidence based guide for navigating the trade offs between cost, speed, and accuracy. We conducted 119 tests on the Deep1M dataset within a resource-constrained 4GB RAM Docker, measuring latency, ingestion speed, storage overhead, and recall accuracy. The results reveal a stark architectural trade-off. ChromaDB delivers highly consistent, low query latency, with only a 1.3-fold performance degradation as data scales. However, this speed comes with significant operational costs: massive storage inefficiency averaging 395 times the raw data size and severe ingestion bottlenecks, showing a 491.7 fold slowdown. Conversely, PostgreSQL with PGVector demonstrates resource efficiency. Its storage overhead is minimal at 3-4 times the raw data size, and it provides 7.0 times better ingestion scalability. Crucially, it achieves statistically superior accuracy at production scale ($\geq 250K$ vectors), delivering near-perfect 99.6-99.8% recall compared to ChromaDB's 91-95%. The trade-off is performance variability, where poorly tuned PostgreSQL queries can be up to 16.6 times slower than ChromaDB. We conclude that for dynamic production applications where TCO, scalability, and high accuracy are priorities, PGVector is more viable. ChromaDB's predictable latency is better suited for latency-critical applications with static data, but only if its high operational costs are acceptable.

I. INTRODUCTION

With the rapid adoption of artificial intelligence, vector databases have transformed into fundamental components in modern system architecture [1], [2], [3], [4]. Its ability to efficiently perform similarity searches on high-dimensional data makes them a key technology for various modern AI applications that fundamentally map 'input vectors' to 'output vectors', such as semantic search, recommendation systems, stock price prediction, and Retrieval-Augmented Generation (RAG) [5], [6], [7], [8]. Although crucial, the various vector database platforms that support this application have quite different server resource consumption profiles and data ingest scalability. This presents system architects with the challenge of choosing the most optimal option that balances operational costs (TCO) and specific architectural requirements, such as the scale of data to be used and server resource limitations [9], [10], [11].

The main dilemma in this selection boils down to two fundamentally different approaches: specialized databases designed for vector management, such as ChromaDB, and generalized databases enriched with vector extensions, such as PostgreSQL with PGVector [12], [13]. This decision often involves a trade-off. Specialized solutions promise unparalleled query speed, as their architecture is purpose-built for Approximate Nearest Neighbor (ANN) search. Conversely, generalized solutions offer the compelling advantage of operational simplicity, allowing teams to manage vector data alongside existing relational data within a single, familiar ecosystem, thereby reducing data silos and integration complexity [14], [15], [16]. However, this strategic

decision is often made without comprehensive comparative evidence, especially in the context of realistic full-stack implementations [12], [17], [18]. Existing studies often focus on comparing algorithms in isolation or are limited to query latency metrics [9], [17]. As a result, there is a significant gap in understanding the holistic impact of these architectural choices, especially on the most decisive aspects in real-world implementations: efficiency and operational costs [18], [19]. Resource usage efficiency, including storage, CPU, and memory, has direct implications for the total cost of ownership (TCO) and long-term scalability of a product. Massive storage overhead, for example, translates directly into higher infrastructure bills and can become an untenable financial burden at production scale. Similarly, high CPU and memory consumption not only increase operational costs but can also lead to resource contention, impacting the performance of other co-located services and creating system instability. This is an aspect that has not been explored in depth in direct comparisons of these two approaches [11], [14], [18]. Furthermore, beyond cost considerations, this architectural choice has other important consequences. The quality and reliability of the system are highly dependent on the accuracy (recall) of search results. In applications like RAG, low recall means the system retrieves irrelevant context, leading directly to model 'hallucinations' and a loss of user trust. A difference of a few percentage points can be a determining factor in risk-sensitive applications [15], [16]. Additionally, the operational agility of the system's ability to quickly absorb new data through the ingestion and indexing process determines the competitiveness and relevance of real-time applications. This 'data freshness' the time from data creation to its availability in search results is critical for real-time applications. A slow ingestion or indexing process creates a data staleness gap, rendering the application uncompetitive [13], [15], [20]. These crucial aspects of accuracy and agility are often secondary considerations in conventional performance analysis. Therefore, this study is designed to fill this gap by presenting a comprehensive benchmark analysis comparing ChromaDB and PostgreSQL PGVector [18], [21]. This study systematically evaluates key performance metrics, including query latency and throughput, resource usage efficiency to quantify its impact on cost, index build time to analyze its influence on system agility, and the overall performance scalability of both solutions. Through evaluation in a controlled environment, this research aims to provide empirical guidance for system architects to navigate the fundamental trade-offs between cost, accuracy, and performance in selecting the most appropriate vector database technology.

As in the analytical framework, this study differs from previous benchmarking studies of "PERFORMANCE ANALYSIS OF CHROMA, QDRANT, AND FAISS DATABASES" [18]. While other studies, such as the analysis of Chroma, Qdrant, and Faiss, focus on latency in a high resource environment using 64 GB of RAM, this research methodology uniquely applies strict single-node resource constraints with only 4 GB of RAM to simulate realistic TCO scenarios. Furthermore, our framework adopts holistic metrics, not only latency and storage size, but also critically analyzes storage overhead and recall accuracy dimensions that are often overlooked in order to fully capture operational trade-offs.

The selection of ChromaDB and PGVector for this study was based on strong architectural justifications, specifically aimed at comparing the philosophies of generalized vs. specialized. PGVector, as a popular extension for PostgreSQL, was chosen to represent a widely used integrated hybrid solution in production scale. Conversely, ChromaDB represents a popular, lightweight, SQLite-based specialized VDB. This makes it the most comparable and relevant counterpart to PGVector, unlike heavyweight VDBs such as Milvus, allowing for a fairer evaluation of TCO and operational trade-offs. For system architects, choosing a VDB is not a search for absolute efficiency, but rather a navigation of contextual trade-offs, particularly between integrated systems such as PGVector and specialized systems such as ChromaDB. Therefore, this study proposes a theoretical underpinning that "efficiency" is a multi-dimensional concept. Efficiency here is defined as a trade-off between competing pillars, including storage cost efficiency, which impacts total cost of ownership computational efficiency, measured by CPU and memory consumption, data agility determined by ingest speed, query capability and accuracy. The choice of architecture between Generalized and Specialized forces different priorities on these dimensions, where the "right" choice depends on the scale of data, as well as feature and resource requirements.

The main novelty of this research lies in its comprehensive head-to-head comparative analysis between ChromaDB and PostgreSQL with the PGVector extension. While most of the existing literature tends to focus on algorithm comparisons or isolated query latency metrics, this research presents a comprehensive understanding that has not been explored in depth before. Specifically, this study presents a full-stack evaluation that measures the direct impact on often overlooked but crucial operational metrics, namely CPU usage, memory consumption, and especially storage overhead. There are still very few studies that provide in-depth empirical comparisons of these efficiency aspects directly between these two popular platforms. Therefore, the main objective of this research is to empirically quantify the architectural trade-off between specialized and generalized VDBs. To achieve this goal, this study systematically evaluates and compares their impact on resource efficiency (TCO), ingest scalability, and recall accuracy, in order to provide evidence-based guidance for system architects.

II. RELATED WORKS/LITERATURE REVIEW

Research on Vector Databases (VDBs) has surged in recent years, driven by the expanding use of high dimensional embeddings in applications like recommendation systems and semantic search. The existing literature

broadly addresses the performance landscape, often categorizing VDBs into specialized, purpose-built systems and generalized systems, typically relational databases augmented with vector search extensions. While numerous studies provide valuable insights, a closer examination reveals specific gaps, particularly concerning comprehensive, full-stack comparisons between certain popular implementations like ChromaDB and PostgreSQL with PGVector.

One major stream of research focuses on integrating vector search capabilities into established relational databases, aiming to leverage existing infrastructure and reduce data silos. The PASE extension for PostgreSQL, for instance, demonstrates the feasibility of incorporating Approximate Nearest Neighbor (ANN) algorithms like IVFFlat and HNSW directly into the database kernel [16]. Their work highlights the potential benefits, such as reduced costs and the ability to handle compound queries that mix vector similarity with traditional relational filters, a crucial requirement for many real world scenarios [16]. However, PASE's analysis also points out inherent trade-offs, such as the significantly longer index construction times associated with graph based methods like HNSW compared to quantization based ones like IVFFlat, impacting system agility [16]. Further exploring the generalized approach, the SingleStore-V system explicitly benchmarks itself against PGVector, providing direct comparative metrics [21]. While claiming superior performance over PGVector in throughput and index build times (noting PGVector required 40.3x to 74.8x more time for HNSW index construction), SingleStore-V's findings underscore the performance challenges faced by generalized VDBs when compared to highly optimized systems, while simultaneously arguing for their necessity due to interoperability needs (e.g., joins, filters) in complex applications [21]. These studies establish PGVector's position within the generalized VDB landscape but often compare it against other generalized systems or highly optimized commercial alternatives, leaving comparisons with lightweight specialized databases less explored.

Contrasting with the generalized approach, specialized vector databases are often designed for specific use cases or deployment models. ChromaDB, for example, is frequently highlighted in the context of Retrieval Augmented Generation (RAG) and local or lightweight deployments [2]. It is characterized as a lightweight, developer friendly option offering native persistence, making it suitable for integration into microservice architectures or for rapid prototyping where persistent storage is key [2]. Studies evaluating systems incorporating Chroma often focus on application level metrics such as retrieval accuracy and end-to-end response latency within conversational AI frameworks [2]. While these studies validate Chroma's utility in specific contexts, they typically do not provide deep, comparative benchmarking against generalized systems like PGVector across a wide range of operational metrics, particularly resource consumption and scalability under load.

Recognizing the limitations of focusing solely on query latency, another body of research emphasizes the need for a broader evaluation framework encompassing metrics critical for production viability. The work on VDB observability provides a crucial conceptual structure, arguing that VDB performance assessment must extend beyond traditional database metrics [1]. It defines key indicators like Recall@K (search quality), ANN Precision (index accuracy versus brute force), and importantly, Index Refresh Latency the time needed to update indices after data ingestion as a direct measure of system agility [1]. This framework also highlights the necessity of monitoring resource utilization (CPU, memory), noting the potential for nonlinear increases under load, which directly impacts operational costs [1]. Complementing this, research focused on I/O efficiency in ANNS stresses the importance of measuring low level costs, especially for large datasets requiring external memory algorithms [17]. Metrics such as I/O costs, running time, pre-processing time (index construction), and index size are identified as essential for analyzing resource overhead and associated costs, reinforcing the idea that ANNS inherently involves a trade-off between efficiency and accuracy [17].

While the previous studies have made important contributions, this literature review summarized in Table 1 reveals a clear research gap. Few studies have comprehensively compared the specialized ChromaDB approach with the generalized PGVector in realistic medium-scale implementation scenarios. In particular, there are almost no studies that conduct holistic and controlled evaluations covering storage overhead (TCO) and recall under strict and identical single-node resource constraints, which is the main justification for this research analytical framework.

TABLE 1
METHODOLOGICAL COMPARISONS AND RESEARCH GAPS

Study	Reviewed Databases	Primary Metrics	Research Gap
"PASE: PostgreSQL Ultra-High-Dimensional Approximate Nearest Neighbor Search Extension" [16].	PASE (IVFFlat, HNSW), Cube, Freddy (IVFADC, IVFADC_PV)	Ingestion build time & Scalability, query Latency & recall, storage Index Size, resource using 251 GB RAM.	Focus on High-Resource environments with 251 GB RAM. Does not analyze TCO in resource-constrained scenarios.
"Performance Analysis Of Chroma, Qdrant, And Faiss Databases" [18].	Chroma, Qdrant & Faiss.	Ingestion insertion time & scalability, query time, database storage size, resource usage 64 GB RAM.	Does not measure Recall accuracy, focuses on High-Resource 64 GB RAM environments.

"SingleStore-V: An Integrated Vector Database System in SingleStore" [21].	SingleStore-V, Milvus & PGVector.	Ingestion index construction, billion-scale scalability, query throughput & recall, resource usage 256 GB RAM	Focus on distributed/billion-scale, focus on high resource environments 256 GB RAM.
--	-----------------------------------	---	---

III. METHODS

This study adopts a quantitative experimental design by conducting a direct comparison (head-to-head benchmarking) between two database systems [14]. This approach allows for objective measurement of various performance metrics under identical and controlled conditions [18], [21]. To ensure this objectivity, process isolation and reproducibility of results, the entire research procedure has been fully automated and executed using structured scripts, minimizing variability and potential bias from human error.

The overall flow of this automated procedure is illustrated in Figure 1. As shown in the diagram, this process is central to the research objective of providing a fair comparison, starting with a strict Docker Environment Setup to ensure identical resource allocation 60% CPU and 4GB RAM for both systems. The flow then iterates through each test configuration, running two distinct measurement phases the Data Ingestion Phase and the Query Phase. During the Data Ingestion Phase, metrics such as total data retrieval time and resource consumption are recorded. After an Index Warm-up step to eliminate bias, the Query Phase then measures critical performance indicators such as latency, throughput (QPS), and Recall, accompanied by additional resource monitoring. This structured process is highly relevant as it ensures that each experiment is reproducible and directly comparable, with all collected data ultimately aggregated for Statistical Analysis to validate the findings.

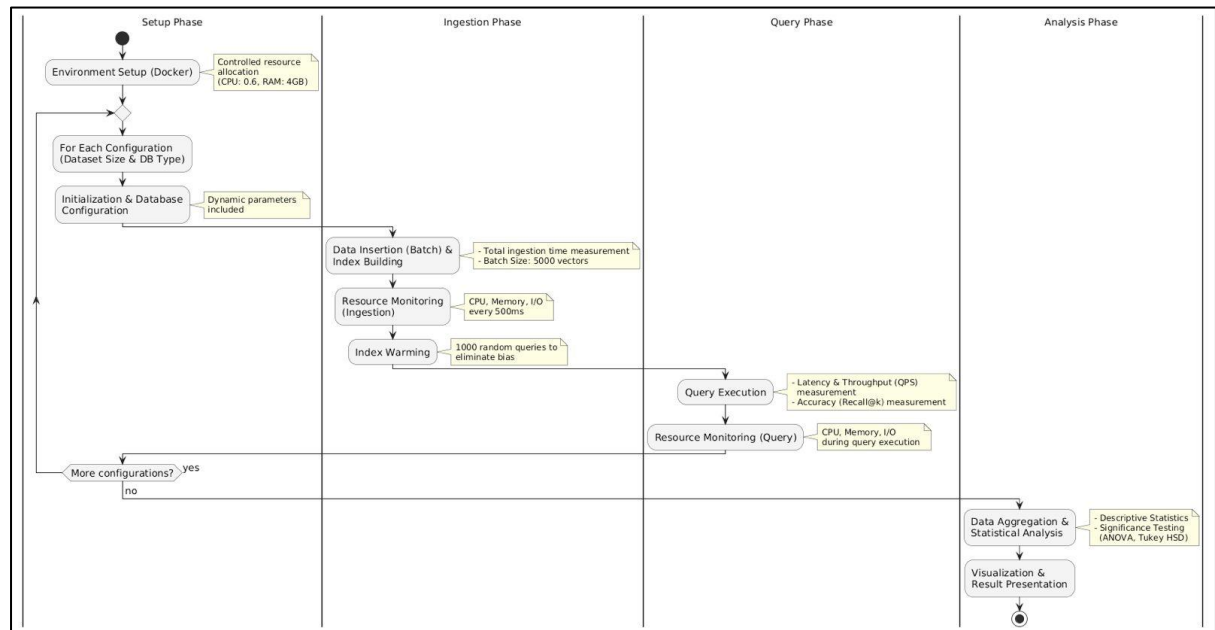


Fig.1 The overall flow of the systematic comparison process.

A. Environment and Experimental Equipment

This experiment was conducted on a single physical machine (MacBook Pro M4 24 GB of Memory) that served as the host for the containerized environment. This approach was chosen not only to simulate modern deployment practices, but also to apply the principle of fairness in resource allocation [22]. To manage this environment, all components, including both databases and monitoring systems, were orchestrated in an integrated manner using Docker Compose.

As the main implementation of this principle of fairness, equal comparison conditions were created by imposing identical and strict resource limit on each database container (ChromaDB and PostgreSQL), namely 0.6 CPU cores (60%) and 4 GB of memory. This deliberate limitation is crucial to simulate real-world scenarios where applications run on limited resources, allowing testing to reveal the actual efficiency of the system.

In this controlled environment, both databases are specifically configured. ChromaDB is set up using industry-standard configurations with L2 squared distance metrics and optimal HNSW index parameters. Meanwhile, the PostgreSQL configuration with the PGVector extension is further optimized, including the activation of `pg_stat_statements` for query monitoring and the use of two index types, HNSW and IVFFlat. Unlike Chroma, the parameters for these two PGVector index types (such as `m`, `ef_construction`, `lists`, and `probes`) are not static, but are dynamically adjusted based on the dataset size to ensure optimal performance at every test scale.

B. Research Materials and Instruments

The main instruments used for validation in this study include benchmark vector datasets and a series of system monitoring software. In terms of data, this study relies on the Deep1M dataset, which is part of the Deep1B benchmark collection [23]. This dataset provides three crucial components: one million base vectors (96 dimensions), ten thousand query vectors, and ground truth data for validation. To evaluate meaningful scalability, testing was conducted incrementally on various subsets, ranging from 10,000 to 1,000,000 vectors.

It is important to note that the selection of Deep1M (one million vectors) instead of the original dataset, Deep1B (one billion vectors), was a deliberate methodological decision. This was based on the fact that the controlled testing environment on a single machine is not designed to handle the scale of data found in Deep1B, which generally requires a distributed computing cluster. Thus, the use of Deep1M allows the research to focus on analyzing the fundamental efficiency of the database architecture in a realistic single-node scenario, without being distorted by hardware limitations. Meanwhile, for the second instrument, a comprehensive monitoring system was built using Prometheus and Grafana running in separate containers. This system collects resource metrics at the system level every 500 milliseconds using `psutil`, while also extracting container-level metrics from Prometheus [24] for detailed resource consumption analysis.

C. Data Collection Procedure

The data collection process for each database configuration and dataset size is carried out automatically and in a structured manner, divided into two main phases: data ingestion and query testing.

In the first phase, namely data insertion and index building, the ingestion process is carried out using a batch method of 5,000 vectors. This approach was chosen to maintain consistency, especially given the limitations of ChromaDB. It is important to note that the time measured covers the entire process, from inserting data into the table to completing index building (build-on-insert), thus providing an overview of the total time required until the data is ready for access.

After the ingestion phase is complete and the data is ready, the process continues to the query testing stage. However, before query measurement begins, an *index* warming protocol is first run with 1,000 random queries. This action aims to eliminate *cold-start* bias and ensure that the data has been loaded into memory. Next, 10,000 queries from the dataset are executed sequentially, where the total completion time is measured to calculate the average latency and *throughput* (QPS). At the same time, the search results from each query are also compared with the ground truth data to calculate accuracy using the Recall@K metric ($k=1, 5$, and 10).

During these two main phases, both data insertion and query testing, a resource monitoring process runs continuously in the background to record CPU usage, memory, and I/O activity in real time.

It should be noted that the reproducibility of this experiment is not controlled by a random seed, but rather ensured by a fully deterministic design. First, the Deep1M dataset used is a deterministic subset created by taking the first 1 million vectors sequentially from the Deep1B dataset, without random sampling or transformation. Second, our methodology uses the entire fixed query set (10,000 vectors) provided by Deep1M for all performance measurements. The 1,000 random queries mentioned earlier are only used during the index warming phase and do not affect the collected metric data. Although this Deep1M dataset (96 dimensions) is ideal for analyzing fundamental architecture efficiency, it is important to note that performance trade-offs, especially those related to the curse of dimensionality, may exhibit different characteristics on higher-dimensional data (e.g., 768-D or 1536-D) that is common in modern transformer models. Therefore, generalizing these findings to very high-dimensional embeddings is a limitation of the scope of this research.

D. Data Analysis Techniques

The raw data collected from the experiments and resource monitoring was then exported into CSV format for further analysis. Descriptive statistical analysis was used to calculate the mean, median, standard deviation, and P95 percentile for each metric. To test the significance of performance differences between groups, a one-way analysis of variance (ANOVA) was used, followed by a Tukey HSD post-hoc test if the results were statistically significant ($p < 0.05$) [25].

IV. RESULTS

This chapter presents empirical results from a comparative performance analysis between ChromaDB and PostgreSQL with the PGVector extension. The data presented is the result of 119 experiments conducted on datasets ranging from 10,000 to 1,000,000 vectors. Data analysis reveals significant and systematic performance differences between the two platforms, particularly in terms of resource efficiency, ingestion speed, and query performance.

A. Resource Efficiency: Fundamental Contrasts in Storage, CPU, and Memory

Resource efficiency analysis revealed fundamental contrasts between the two databases, particularly in three key aspects, namely storage, CPU utilization, and memory usage.

The most significant findings emerged in storage efficiency, where the analysis revealed two fundamentally different paradigms. On the one hand, PostgreSQL with PGVector demonstrated remarkable efficiency, with storage costs increasing linearly and predictably as the dataset size grew. Conversely, the most extreme storage inefficiency is seen in ChromaDB, a contrast that is now detailed in Table 2. The most striking example of this inefficiency is in the 10,000 vector dataset, where ChromaDB recorded an overhead ratio of 1,554.78x, consuming 5.56 GB for data that should only require 3.66 MB in raw form. This pattern of massive overhead at small scales, summarized in Table 2, suggests that ChromaDB's architectural design likely prioritizes query performance over storage savings, with the overhead ratio decreasing to 18.39x at 1 million vectors as the proportional impact of fixed storage costs diminishes.

TABLE 2
STORAGE EFFICIENCY COMPARISON ACROSS DATASET SCALES

Dataset Size (Vectors)	Raw Data Size (MB)	ChromaDB HNSW Overhead (Avg)	PGVector HNSW Overhead (Avg)	PGVector HNSW Overhead (Avg)	Efficiency Gap
10,000	3,66	1,554.78x	5.11x	4.66x	304x
25,000	9,16	623.39x	3.85x	3.22x	162x
50,000	18,31	313.17x	3.42x	2.74x	92x
100,000	36,62	158.06x	3.53x	2.50x	45x
250,000	94,55	64.71x	3.40x	2.36x	19x
500,000	183,11	33.83x	3.60x	2.31x	9x
1,000,000	366,21	18.39x	4.25x	2.28x	4x

The overhead ratio comparison data, summarized in Table 3, indicates that ChromaDB's design likely prioritizes query performance over storage savings, as evidenced by its significant initial storage footprint even on the smallest dataset.

TABLE 3
COMPARISON OF OVERHEAD EFFICIENCY

INDEX TYPE	95% CIs	AVERAGE RATIO OF OVERHEAD
POSTGRESQL (HNSW)	3.54 TO 4.2	3.88X OF THE RAW DATA
POSTGRESQL (IVFFLAT)	2.39 TO 3.34	2.87X OF THE RAW DATA
CHROMADB (HNSW)	85.81 TO 704.57	395.19X OF THE RAW DATA

Statistical validation through one-way ANOVA confirmed significant differences in storage overhead ($F=7.49$, $p=0.002$). Given the non-normal distribution of this metric, the non-parametric Kruskal-Wallis test was performed, reinforcing this finding ($H=32.55$, $p<0.001$) and revealing a large effect size ($\epsilon^2=0.78$), which indicates database configuration explains 78.3% of the variance. Tukey HSD post-hoc tests revealed ChromaDB significantly differed from both PostgreSQL PGVector HNSW with mean difference = 391.31x, $p=0.005$ and IVFFlat with mean difference = 392.32x, $p=0.005$, while PostgreSQL configurations showed no significant difference (mean difference = 1.01x, $p=1.000$). The 95% confidence intervals in Table 3 above demonstrate measurement precision, with PostgreSQL showing narrow intervals reflecting consistent performance and ChromaDB exhibiting wider intervals indicating greater variability, particularly at smaller dataset sizes.

Turning to CPU and memory utilization, different patterns emerge depending on the phase, as shown in Figure 2 for CPU and Figure 3 for memory. At first glance, the line charts for the ingestion phase in both figures appear highly variable, with performance lines for ChromaDB and PostgreSQL frequently crisscrossing. This 'noise' is an expected and important artifact of the experimental design, as these charts plot the performance results from all 119 different test configurations (e.g., varying HNSW m values, IVFFlat lists values) simultaneously against dataset size. Statistical validation through one-way ANOVA revealed no significant differences in CPU utilization

($F=0.56, p=0.571$), a finding confirmed by the non-parametric Kruskal-Wallis test ($H=1.15, p=0.564$), indicating comparable computational demands. For memory usage, while ANOVA initially suggested a modest difference ($F=3.40, p=0.037$), the more robust Kruskal-Wallis test showed no significant difference ($H=1.33, p=0.515$). We therefore conclude there are no statistically significant differences in either CPU or memory (GB) demands, as the ANOVA result was likely a false positive due to data skew. The 95% confidence intervals reinforce this finding, showing highly overlapping performance: ChromaDB at 25.65 to 28.06% CPU and 7.34 to 7.53 GB memory, PostgreSQL HNSW at 26.08 to 30.07% CPU and 7.43 to 7.70 GB memory, and PostgreSQL IVFFlat at 26.23 to 29.14% CPU and 7.36 to 7.44 GB memory.

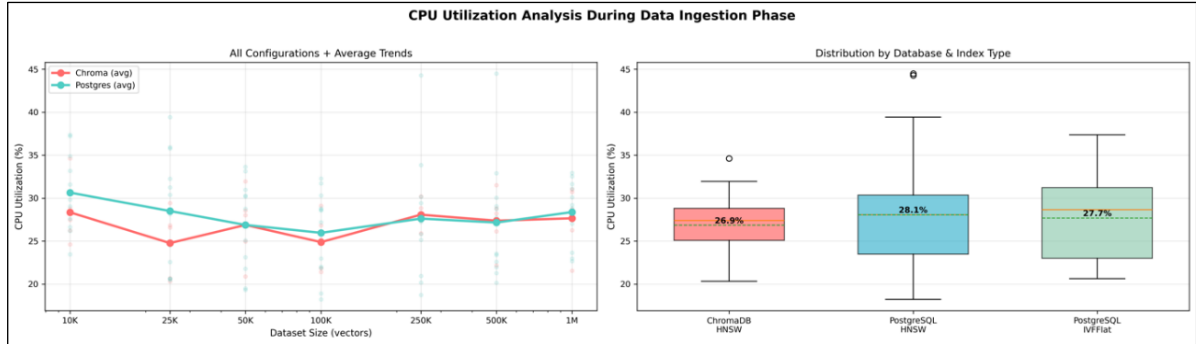


Fig. 2 CPU utilization patterns during data ingestion show nearly identical average consumption (26.9% vs 27.9%) across 119 test configurations, despite visible variability from different index-building parameters.

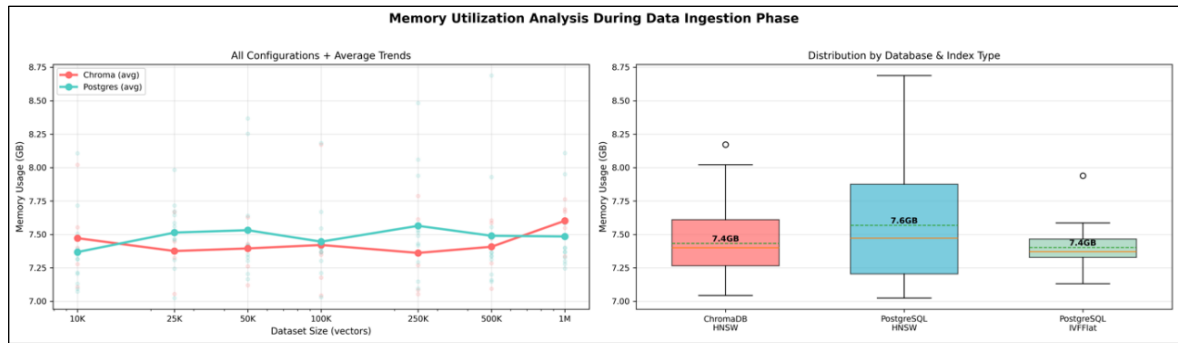


Fig. 3 Memory utilization during data ingestion demonstrates comparable consumption patterns 7.4 GB vs 7.5 GB across both databases, reinforcing that storage efficiency differences are architectural rather than computational.

B. Ingestion Performance and Index Building Scalability

Data ingestion performance analysis shows significant advantages for PostgreSQL with PGVector, as visualized in Figure 4. This log-scale visualization displays the insertion times of all 119 experimental configurations. As shown by the graph, the orange PGVector line shows insertion times that are generally more consistent and lower than the blue ChromaDB line. Many PostgreSQL configurations managed to maintain better performance even when the dataset size increased from 10K to 1M vectors. In contrast, the sharp upward trajectory of the ChromaDB performance line clearly illustrates the problem of super-linear scaling, which makes it a bottleneck for production-scale ingest workloads.

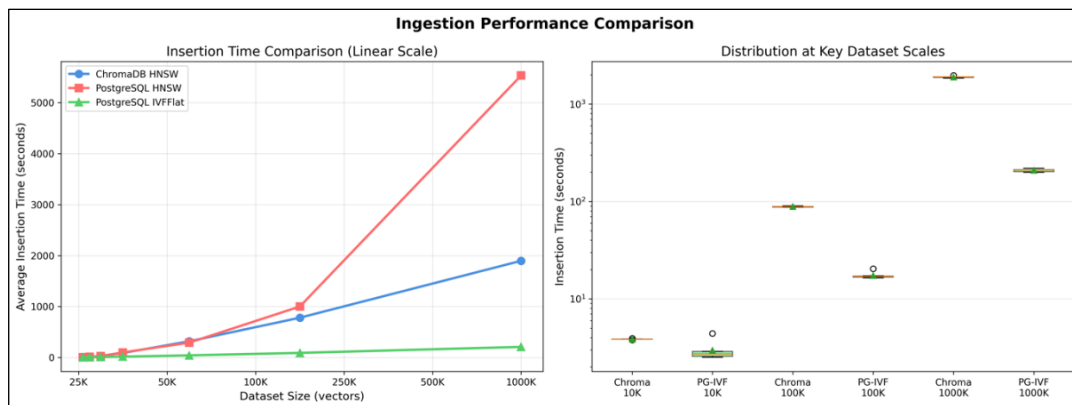


Fig. 4 Ingestion performance comparison across 119 configurations reveals PostgreSQL's superior scalability, with IVFFlat achieving 7.0x better scaling efficiency than ChromaDB when dataset size increases from 10K to 1M vectors.

To quantify these scalability differences in more detail, Table 4 summarizes the average ingest time at each scale. The table shows a clear performance hierarchy: the IVFFlat index on PostgreSQL provides the fastest ingest time at every dataset scale. The most significant finding is the drastic difference in performance degradation when the dataset size is increased 100-fold from 10,000 to 1,000,000 vectors. ChromaDB's ingest time slowed by a factor of 493.6x from 3.85 seconds to 1,900.79 seconds. In stark contrast, PostgreSQL's IVFFlat index only slowed by a factor of 70.1x from 2.51 seconds to 199.36 seconds. This striking difference, where ChromaDB is 6.2x less scalable than PGVector IVFFlat, indicates that ChromaDB exhibits super-linear scaling characteristics, which become a severe performance bottleneck at larger data scales.

TABLE 4
DETAILED BREAKDOWN BY DATASET SIZE

Dataset Size	PostgreSQL IVFFlat (Avg)	PostgreSQL HNSW (Avg)	ChromaDB HNSW (Avg)	Performance Ratio
10,000	2.98 s	6.47 s	3.86 s	1.3x Faster
25,000	5.30 s	13.86 s	11.94 s	2.3x faster
50,000	9.77 s	29.17 s	32.21 s	3.3x faster
100,000	17.39 s	99.32 s	88.67 s	5.1x faster
250,000	42.29 s	291.22 s	318.18 s	7.5x faster
500,000	90.81 s	1000.33 s	780.91 s	8.6x faster
1,000,000	208.63 s	5533.60 s	1897.25 s	9.1x faster
Scaling Factor (10K to 1M)	70.1x	854.7x	491.7x	7.0x better scaling

Statistical validation through ANOVA confirmed significant ingestion performance differences with $F = 6.39$ and $p = 0.002$, a finding reinforced by the non-parametric Kruskal-Wallis test with $H = 15.15$ and $p < 0.001$. Both tests demonstrated a comparable medium effect size, with ANOVA $\eta^2 = 0.10$ and Kruskal-Wallis $\epsilon^2 = 0.11$, explaining approximately 10-11% of performance variance. Tukey HSD post-hoc analysis revealed significant difference only between PostgreSQL HNSW and IVFFlat with $p = 0.002$ and mean difference of 942.40 seconds, while ChromaDB showed no significant difference from either PostgreSQL configuration. The 95% confidence intervals quantify performance consistency, with IVFFlat demonstrating narrow ranges of 31.98 to 75.78 seconds overall, contrasting with HNSW's wider intervals of 388.72 to 1603.84 seconds for PostgreSQL and 222.51 to 672.64 seconds for ChromaDB, indicating greater variability in HNSW implementations.

C. Query Performance and Accuracy

Query performance analysis focuses on three key metrics latency, recall accuracy, and performance consistency. The detailed analysis visualized in Figure 5 reveals a fundamental architectural trade-off through three complementary views. The left panel shows performance across search depths ($k=1, 5, 10$), the center panel illustrates performance variability through coefficient of variation, and the right panel presents the recall latency trade-off. Together, these visualizations demonstrate two contrasting design philosophies. ChromaDB (blue markers) exhibits remarkably consistent and predictable query latency, with minimal degradation across scales. Its average query time increases only 1.3x from 22.25 seconds at 10K vectors to 28.47 seconds at 1M vectors, maintaining stable performance regardless of dataset size. Conversely, PostgreSQL (orange markers) demonstrates extreme performance variability. At small scales $\leq 50K$ vectors, optimized IVFFlat configurations can achieve query times as low as 5.15 seconds, making them up to 4.3x faster than ChromaDB. However, at production scales $\geq 250K$ vectors, PostgreSQL's performance becomes highly configuration-dependent, with some suboptimal HNSW configurations degrading to 473.65 seconds at 1M vectors approximately 16.6x slower than ChromaDB at the same scale.

Statistical validation through ANOVA confirmed significant query latency differences with $F = 16.74$ and $p < 0.001$. This finding was reinforced by the non-parametric Kruskal-Wallis test with $H = 17.45$ and $p < 0.001$, although the more robust, non-parametric effect size was small at $\epsilon^2=0.04$, compared to the medium effect of $\eta^2=0.09$ suggested by ANOVA. Tukey HSD post-hoc analysis revealed ChromaDB differed significantly from PostgreSQL IVFFlat with $p < 0.001$ and mean difference of 52.71 seconds, and PostgreSQL HNSW differed from IVFFlat with $p < 0.001$ and mean difference of 35.12 seconds. The 95% confidence intervals demonstrate ChromaDB's consistency at 23.77 to 25.03 seconds, contrasting with PostgreSQL HNSW's wider range of 29.62

to 54.34 seconds and IVFFlat's 60.02 to 94.19 seconds, reflecting greater variability in PostgreSQL implementations.

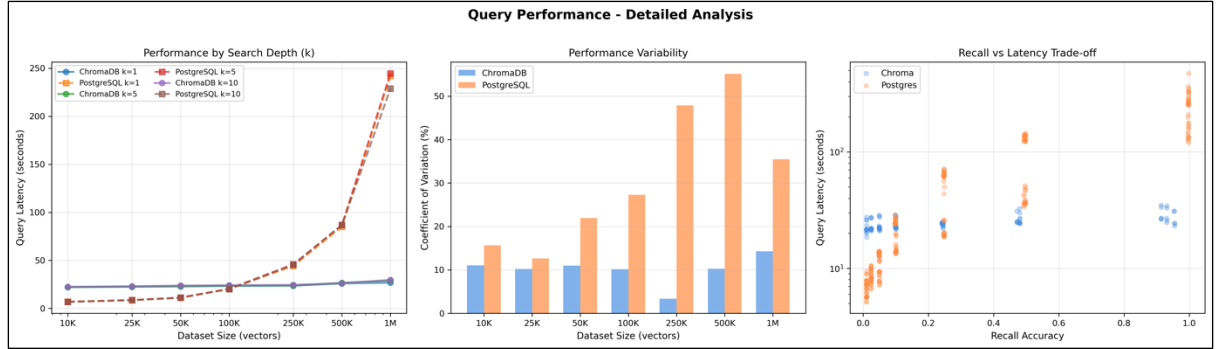


Fig. 5 Query performance detailed analysis reveals ChromaDB's consistent 1.3x degradation (22.25s to 28.47s) versus PostgreSQL's extreme variability (5.15s to 473.65s), demonstrating the trade-off between configuration complexity and predictable performance at scale..

As a direct consequence of this low and consistent latency, ChromaDB maintains more predictable query performance at every scale tested. In contrast, although PostgreSQL can achieve faster query times at small scales when properly configured, its performance is highly configuration-dependent and becomes less predictable at large scales. The comparative analysis moves on to accuracy, which shows that PostgreSQL's recall advantage emerges progressively with scale. On small datasets $\leq 100K$ vectors, both databases show largely comparable recall performance with mixed statistical results, as detailed in Table 5. However, at production scale $\geq 250K$ vectors, PostgreSQL demonstrates consistently higher recall accuracy at all k values with highly significant differences ($p < 0.001$). The accuracy findings at the 1 million vector scale, summarized in Table 6, confirm PostgreSQL's clear superiority, achieving near perfect recall rates 99.6-99.8% compared to ChromaDB's 91.4-95.4%.

TABLE 5
RECALL COMPARISON ACROSS DATASET SCALES

Dataset Size	Recall@1 Ratio	Recall@5 Ratio	Recall@10 Ratio
10k Vectors	1.00x (equal)	1.00x (equal)	1.00x (equal)
25k Vectors	1.00x (equal)	1.00x (equal)	1.00x (equal)
50k Vectors	1.00x (equal)	1.00x (equal)	1.00x (equal)
100k Vectors	1.00x (equal)	1.00x (equal)	1.00x (equal)
250k Vectors	1.00x (equal)	1.02x	1.02x
500k Vectors	1.03x	1.04x	1.05x
1M Vectors	1.05x	1.07x	1.09x

TABLE 6
RECALL PERFORMANCE AT SCALE (1M VECTORS)

Metric	PGVector Recall	ChromaDB Recall	Significance (p-value)	Accuracy Ratio
Recall@1	0.9981	0.9542	$p < 0.001$	1.05x
Recall@5	0.9972	0.9308	$p < 0.001$	1.07x
Recall@10	0.9963	0.9138	$p < 0.001$	1.09x

Statistical validation through one-way ANOVA with $F = 0.05$ and $p = 0.950$, along with the non-parametric Kruskal-Wallis test with $H = 0.66$ and $p = 0.721$, both confirmed no significant overall differences in recall accuracy. Both tests also indicated a negligible effect size with $\eta^2 = 0.0003$ and $\epsilon^2 = -0.004$, explaining essentially 0% of variance across all configurations. However, scale-dependent t-tests showed distinct patterns with small datasets yielding mixed results where 6 of 12 comparisons achieved significance, while large datasets demonstrated uniformly significant differences with all 9 tests at scales $\geq 250K$ showing $p < 0.001$. The 95% confidence intervals demonstrate comparable uncertainty across systems with ChromaDB at 0.202 to 0.324, PostgreSQL HNSW at 0.216 to 0.334, and PostgreSQL IVFFlat at 0.215 to 0.334, confirming that while overall recall distributions overlap substantially, PostgreSQL achieves systematically higher accuracy at production scales through superior index quality rather than fundamental architectural differences.

V.DISCUSSION

The results of this experiment demonstrate a fundamental trade-off between two vector database system paradigms, namely specialized (ChromaDB) and generalized (PostgreSQL). This finding is in line with the

dichotomy of modern vector system design as described by [21]. In this context, ChromaDB, as a native open source system, exhibits very stable query speed with low latency thanks to Approximate Nearest Neighbor (ANN)-based index optimization, as shown by [18]. However, these advantages come with the consequences of high storage overhead and significantly slowed ingest performance, a pattern also found in memory-centric systems such as Faiss.

Conversely, PostgreSQL systems with extensions such as PGVector and PASE are designed to provide interoperability and integration of vector and non-vector data within a single relational ecosystem. This approach, as explained by [14], enables full support for SQL and ACID consistency. However, because the PostgreSQL architecture is disk storage oriented, some raw query performance must be sacrificed. Our results empirically validate this trade-off: PostgreSQL's query performance showed extreme variability at Figure 5, with some configurations running 16.6x slower than ChromaDB at Figure. Nevertheless, our findings prove this sacrifice yields significant operational benefits: exceptional resource efficiency a 1.29x average overhead at Table 2 and vastly superior ingestion scalability 7.0x better scaling than ChromaDB at Table 3, reinforcing its stability for complex hybrid queries [21].

These findings reinforce the argument put forward by [14], which identifies the structural limitations of PostgreSQL's relational architecture in managing high-dimensional vector data. Although extensions such as PASE [16] and PGVector have integrated ANN algorithms, their efficiency remains limited by ACID consistency requirements. We observed this limitation not as a failure, but as extreme performance variability at Figure 5, where performance is highly dependent on indexing parameters. On the other hand, while a study by [18] shows memory based systems like Faiss excel in latency, our results for ChromaDB at Table 3 demonstrate that even disk-based specialized systems face severe scalability challenges, occupying a middle ground that does not fully escape the performance penalties of data growth.

Architecturally, this difference reflects two contrasting optimization philosophies. Systems such as ChromaDB follow the principle of "one-size-does-not-fit-all" [15], [21]. Our results validate this, showing it is optimized for one metric query speed at Figure 5, the catastrophic expense of others 395x storage overhead at Table 2. Conversely, PostgreSQL advocates a "one-size-fits-all" approach that emphasizes interoperability [12], [21]. Our findings support this, demonstrating it provides a "good enough" holistic balance: statistically superior recall at scale as explained in Table 5, elite storage efficiency at Table 2, and sustainable ingestion at Table 3, confirming it is a more viable integrated solution.

The architectural implications of these findings are significant. ChromaDB's 395x average storage overhead and 491.7x ingest scalability degradation directly translate to higher TCO and operational bottlenecks. However, this is the explicit trade-off for its primary advantage: highly consistent query performance. Its 1.3x latency degradation offers a predictable speed that PostgreSQL's variable, 16.6x slower performance cannot guarantee, making ChromaDB a superior choice for user-facing applications where predictable response time is more critical than storage cost.

The novelty of this research lies in the direct evaluation of two cross-paradigm systems in a single-node environment with a medium-sized dataset. Unlike previous studies that focused more on comparing PostgreSQL PASE with Faiss or Milvus, this research presents an analysis of ChromaDB, which is now an important component in the Retrieval-Augmented Generation (RAG) pipeline as described by [19]. Thus, this research enriches the understanding of the relevance of PostgreSQL for enterprise needs that demand consistency and relational integration, and confirms the superiority of ChromaDB for fast inference scenarios and local integration in LLM applications.

Therefore, the results of this study not only confirm previous empirical findings, but also broaden the discourse on performance and efficiency limits between vector database system paradigms. Future research could focus on exploring distributed architectures and applying adaptive index configurations to bridge the gap between specialized performance and generalized reliability more effectively.

VI. CONCLUSIONS

For architects prioritizing low total cost of ownership, high data accuracy, and maximum accuracy typical of core production systems with dynamic data, PostgreSQL is the sustainable choice. Four times storage efficiency, seven times data retrieval scalability, and, critically, a superior statistical recall rate of 99.6-99.8% at scale are the determining factors. However, the explicit trade-off is query performance risk a 16.6x latency degradation requires expert tuning to avoid severe bottlenecks. Conversely, for applications where consistent and fast query latency is the primary metric, such as applications interacting with users using static data like RAG on fixed documents, ChromaDB is the performance specific choice. Its 1.3x latency degradation offers predictable speed, which PostgreSQL cannot guarantee. This speed, however, comes with explicit trade-offs in the form of extremely high storage total cost of ownership (395x), poor data retrieval scalability (491.7x), and significantly lower recall accuracy (91-95%) at production scale.

The contribution of this research extends beyond its specific findings to the methodological blueprint it provides. This study's replicable, holistic framework moves beyond isolated algorithm evaluations by explicitly

quantifying the often-overlooked "hidden costs" of TCO, such as storage overhead. This full-system perspective, which is more relevant to real world implementation practices, can thus serve as a model for future research, providing a blueprint for evaluating the complex, multidimensional trade-offs in other database systems.

Several limitations in this study directly guide future work. First, all experiments were conducted in a single-node environment. A logical next step would be to investigate how these performance trade-offs manifest in distributed architectures, where horizontal scaling can alter cost dynamics. Second, there is a methodological note regarding the measurement of Recall@K. Although our relative comparisons remain statistically valid, addressing potential inconsistencies in ID mapping remains an important task. Finally, this research focuses on 96-D data. Future work should analyze the impact on higher-dimensional datasets (e.g., 768-D) common in modern transformer models, as well as explore advanced hybrid indexing techniques to bridge the gap between PostgreSQL's efficiency and ChromaDB's query speed.

REFERENCES

- [1] D. Hemantbhai Patel, "Vector Databases and Embedding Stores for Pipeline Observability," *Int. J. Res. Trends Innov.*, vol. 10, no. 9, Sep. 2025, doi: 10.56975/ijrti.v10i9.206710.
- [2] Dheerendra Yaganti, "Enterprise-Grade Conversational Intelligence: A Domain-Aware Chatbot Framework using Gpt-3.5, Langchain, And Rag with Local Vector Indexing," *Int. J. Adv. Res. Sci. Commun. Technol.*, vol. 4, no. 1, pp. 625–631, May 2024, doi: 10.48175/IJARSCT-18099.
- [3] Aradhya Kc and Divya TI, "Vector Databases," *Open Access Res. J. Eng. Technol.*, vol. 7, no. 1, pp. 096–104, Sep. 2024, doi: 10.53022/oarjet.2024.7.1.0043.
- [4] A. Jain and S. Pragati, "Empowering Identity Resolution with Vector Databases," *INTERANTIONAL J. Sci. Res. Eng. Manag.*, vol. 09, no. 02, pp. 1–9, Feb. 2025, doi: 10.55041/IJSREM41864.
- [5] Sarat Kiran, "Hybrid Retrieval-Augmented Generation (RAG) Systems with Embedding Vector Databases," *Int. J. Sci. Res. Comput. Sci. Eng. Inf. Technol.*, vol. 11, no. 2, pp. 2694–2702, Mar. 2025, doi: 10.32628/CSEIT25112702.
- [6] Siva Prakash, "The Role of Cloud-Based Vector Databases and Retrieval Augmented Generation (RAG) for Generative AI in Financial Markets Analysis," *World J. Adv. Eng. Technol. Sci.*, vol. 15, no. 3, pp. 1609–1618, Jun. 2025, doi: 10.30574/wjaets.2025.15.3.1034.
- [7] W. Jin, N. Wang, L. Zhang, X. Tian, B. Shi, and B. Zhao, "A Review of AI-Driven Automation Technologies: Latest Taxonomies, Existing Challenges, and Future Prospects," *Comput. Mater. Contin.*, vol. 84, no. 3, pp. 3961–4018, Jul. 2025, doi: 10.32604/cmc.2025.067857.
- [8] A. Hermawan, S. Ananda, and Junaedi, "Enhancing Stock Price Forecasting: Optimizing Neural Networks with Moving Average Data," *Bit-Tech*, vol. 7, no. 3, pp. 798–808, Apr. 2025, doi: 10.32877/bt.v7i3.2196.
- [9] Z. Zhu, "Strategies for Improving Vector Database Performance through Algorithm Optimization," *Sci. J. Technol.*, vol. 7, no. 2, pp. 138–144, Feb. 2025, doi: 10.54691/eacstn55.
- [10] Rajesh Daruvuri, "Harnessing vector databases: A comprehensive analysis of their role across industries," *Int. J. Sci. Res. Arch.*, vol. 7, no. 2, pp. 703–705, Dec. 2022, doi: 10.30574/ijrsra.2022.7.2.0334.
- [11] T. Taipalus, "Vector database management systems: Fundamental concepts, use-cases, and current challenges," *Cogn. Syst. Res.*, vol. 85, p. 101216, Jun. 2024, doi: 10.1016/j.cogsys.2024.101216.
- [12] J. J. Pan, J. Wang, and G. Li, "Survey of vector database management systems," *VLDB J.*, vol. 33, no. 5, pp. 1591–1615, Sep. 2024, doi: 10.1007/s00778-024-00864-x.
- [13] J. Wang, E. Hanson, G. Li, Y. Papakonstantinou, H. Simhadri, and C. Xie, "Vector Databases: What's Really New and What's Next? (VLDB 2024 Panel)," *Proc. VLDB Endow.*, vol. 17, no. 12, pp. 4505–4506, Aug. 2024, doi: 10.14778/3685800.3685911.
- [14] Y. Zhang, S. Liu, and J. Wang, "Are There Fundamental Limitations in Supporting Vector Data Management in Relational Databases? A Case Study of PostgreSQL," in *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, Utrecht, Netherlands: IEEE, May 2024, pp. 3640–3653. doi: 10.1109/ICDE60146.2024.00280.
- [15] C. Wei *et al.*, "AnalyticDB-V: a hybrid analytical engine towards query fusion for structured and unstructured data," *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 3152–3165, Aug. 2020, doi: 10.14778/3415478.3415541.
- [16] W. Yang, T. Li, G. Fang, and H. Wei, "PASE: PostgreSQL Ultra-High-Dimensional Approximate Nearest Neighbor Search Extension," in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, Portland OR USA: ACM, Jun. 2020, pp. 2241–2253. doi: 10.1145/3318464.3386131.
- [17] M. Li, Y. Zhang, Y. Sun, W. Wang, I. W. Tsang, and X. Lin, "I/O Efficient Approximate Nearest Neighbour Search based on Learned Functions," in *2020 IEEE 36th International Conference on Data Engineering (ICDE)*, Dallas, TX, USA: IEEE, Apr. 2020, pp. 289–300. doi: 10.1109/ICDE48307.2020.00032.
- [18] E. Öztürk and A. Mesut, "Performance Analysis Of Chroma, Qdrant, And Faiss Databases," *UNITECH – Sel. Pap.*, 2024, doi: 10.70456/TBRN3643.

- [19] C. Jeong, "A Study on the Implementation of Generative AI Services Using an Enterprise Data-Based LLM Application Architecture," *Adv. Artif. Intell. Mach. Learn.*, vol. 03, no. 04, pp. 1588–1618, Oct. 2023, doi: 10.54364/AAIML.2023.1191.
- [20] S. Kale, "From Text to Recommendations: How Vector Databases are Revolutionizing Personalized Content Delivery," *Int. J. Res. Appl. Sci. Eng. Technol.*, vol. 12, no. 4, pp. 3376–3387, Apr. 2024, doi: 10.22214/ijraset.2024.59852.
- [21] C. Chen *et al.*, "SingleStore-V: An Integrated Vector Database System in SingleStore," *Proc. VLDB Endow.*, vol. 17, no. 12, pp. 3772–3785, Aug. 2024, doi: 10.14778/3685800.3685805.
- [22] V. Walter-Tscharf, "Practical Approach to Monitor Runtime Engine Statistics for Apache Spark and Docker Swarm using Graphite, Grafana, and CollectD," in *2021 International Conference on Converging Technology in Electrical and Information Engineering (ICCTEIE)*, Bandar Lampung, Indonesia: IEEE, Oct. 2021, pp. 91–96. doi: 10.1109/ICCTEIE54047.2021.9650649.
- [23] J. Liu *et al.*, "Optimizing Graph-based Approximate Nearest Neighbor Search: Stronger and Smarter," in *2022 23rd IEEE International Conference on Mobile Data Management (MDM)*, Paphos, Cyprus: IEEE, Jun. 2022, pp. 179–184. doi: 10.1109/MDM55031.2022.00045.
- [24] Professor, Department of Communication Systems Engineering, University of Science and Technology, Khartoum, Sudan and M. D. Elradi, "Prometheus & Grafana: A Metrics-focused Monitoring Stack," *J. Comput. Allied Intell.*, vol. 3, no. 3, pp. 28–39, Jun. 2025, doi: 10.69996/jcai.2025015.
- [25] A. Nanda, B. Mohapatra, A. Mahapatra, A. Mahapatra, and Mahapatra Abinash, "Multiple comparison test by Tukey's honestly significant difference (HSD): Do the confident level control type I error," *Int. J. Stat. Appl. Math.*, vol. 6, no. 1, pp. 59–65, 2021, doi: 10.22271/math.2021.v6.i1a.636.